

CSCI 423

Introduction to Algorithms



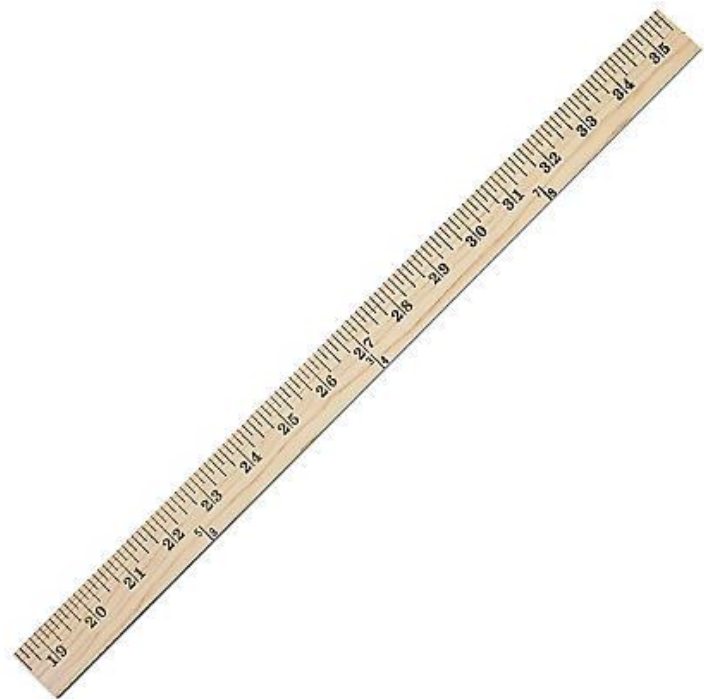
DR. RAAFAT ELFOULY

Program Efficiency

- Our primary concern is EFFICIENCY.
- We want efficient programs.
- How do we measure the efficiency of a program?
(Assume we are concerned primarily with **time**.)
 - On what input?
 - How do we speed it up?
 - When do we stop speeding it up?
 - Should we bother with writing the program in the first place?

Algorithm Efficiency

- Since we don't want to write worthless programs, we will focus on algorithm efficiency.
- We need a yardstick.



Algorithm Efficiency

- Since we don't want to write worthless programs, we will focus on algorithm efficiency.
- We need a yardstick.
 - It should measure something we care about.
 - It should be **quantitative**, allowing comparisons.
 - It should be easy to compute (the measure, not the program).
 - It should be a good predictor

Algorithm Efficiency

- We need:
 - A measure for problem size.
 - A measure for solution effort.
 - Use key operations as a measure of solution effort.
 - Total cost is a function of problem size and key operations.

Cost Model

- To get a measurement, we need a model.
- Example:
 - Assigning to a variable takes fixed time.

`sum = n*n;`

One assignment was made, so the cost is 1.

Cost Model

- Example:

sum = 0;

for (i=1; i<=n; i++)

sum = sum + n;

Assignments made are

$$1 + \sum_{i=1}^n 1 = 1 + n$$

Cost Model

- Example:

```
sum = 0;  
for (i=1; i<=n; i++)  
  for (j=1; j<=n; j++)  
    sum = sum + 1;
```

Assignments made are

$$1 + \sum_{i=1}^n \sum_{j=1}^n 1 = 1 + n^2 = n^2 + 1$$

Big Issues

- How do we create an efficient algorithm?
 - Q: How do we recognize a “good” algorithm?
 - A: By the relationship of its performance to the intrinsic difficulty of the problem.

General Plan

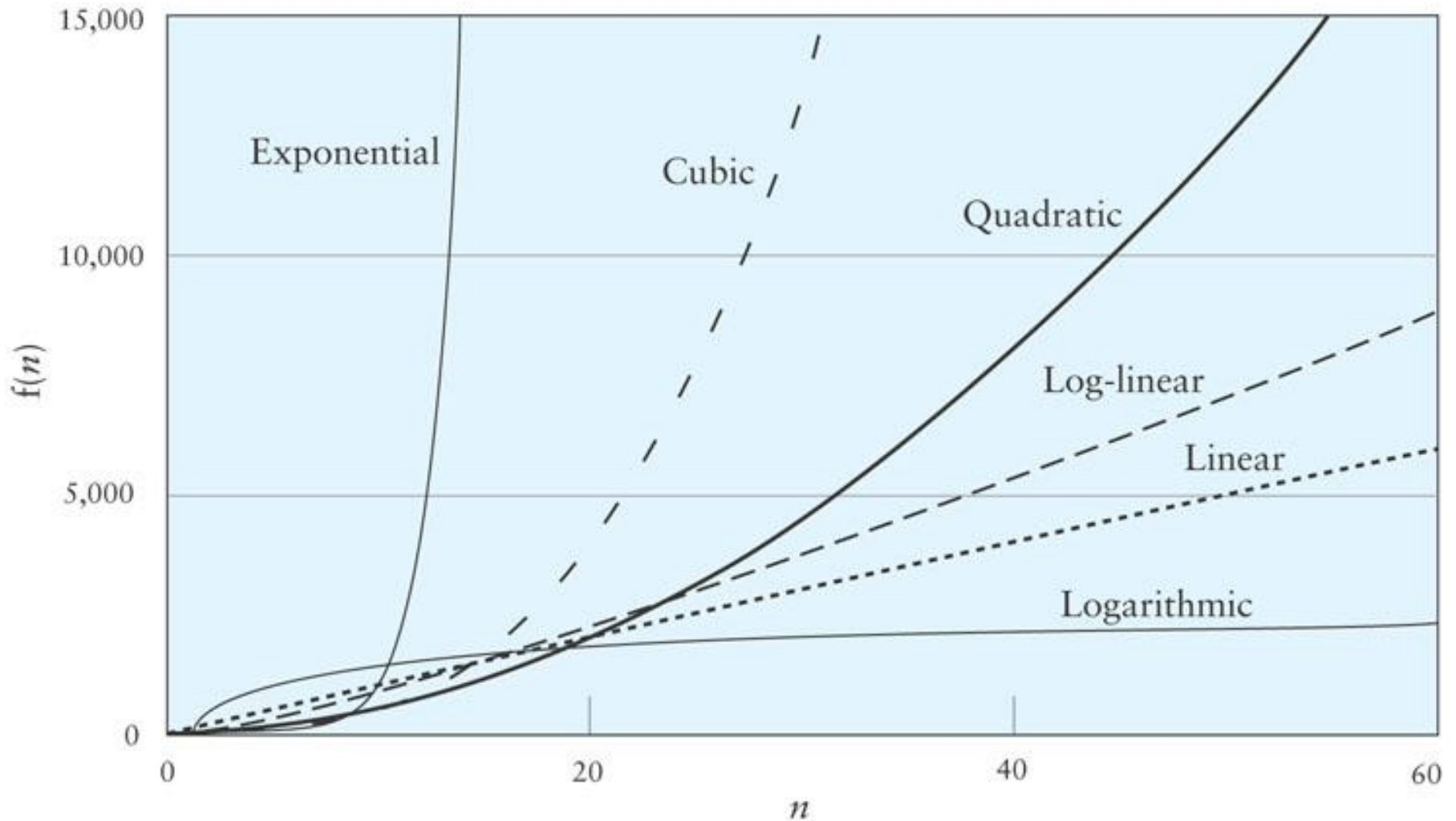
- Define a PROBLEM.
- Build MODEL to measure cost of solution to problem.
- Design an ALGORITHM to solve the problem.
- ANALYZE both the problem and the algorithm under the model to get the lower and upper bounds.
- COMPARE the bounds to see if our solution is “good enough”.

Order of growth

11

- Most important: Order of growth within a constant multiple as $n \rightarrow \infty$
- Example:
 - How much longer does it take to solve problem of double input size?

Different Growth Rates



Effects of Different Growth Rates

$O(f(n))$	$f(50)$	$f(100)$	$f(100)/f(50)$
$O(1)$	1	1	1
$O(\log n)$	5.64	6.64	1.18
$O(n)$	50	100	2
$O(n \log n)$	282	664	2.35
$O(n^2)$	2500	10,000	4
$O(n^3)$	12,500	100,000	8
$O(2^n)$	1.126×10^{15}	1.27×10^{30}	1.126×10^{15}
$O(n!)$	3.0×10^{64}	9.3×10^{157}	3.1×10^{93}

Algorithms with Exponential and Factorial Growth Rates

- Algorithms with exponential and factorial growth rates have an effective **practical limit on the size of the problem** they can be used to solve
- With an $O(2^n)$ algorithm, if 100 inputs takes an hour then,
 - ▣ 101 inputs will take 2 hours
 - ▣ 105 inputs will take 32 hours
 - ▣ **114 inputs will take 16,384 hours (almost 2 years!)**

Values of some important functions as $n \rightarrow \infty$

15

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms

Asymptotic order of growth

16

A way of comparing functions that ignores:
constant factors and **small input sizes**

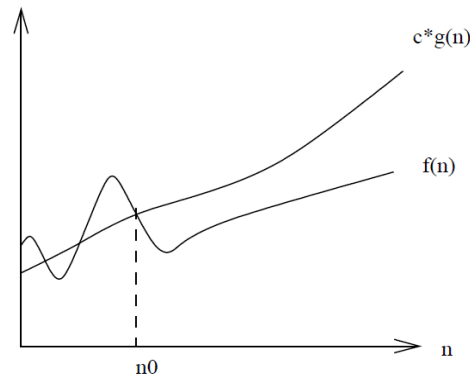
- $O(g(n))$: class of functions $f(n)$ that grow no faster than $g(n)$
 - $g(n) \geq f(n)$
- $\Theta(g(n))$: class of functions $f(n)$ that grow at same rate as $g(n)$
 - $c1 \cdot g(n) \geq f(n) \geq c2 \cdot g(n)$
- $\Omega(g(n))$: class of functions $f(n)$ that grow at least as fast as $g(n)$
 - $g(n) \leq f(n)$



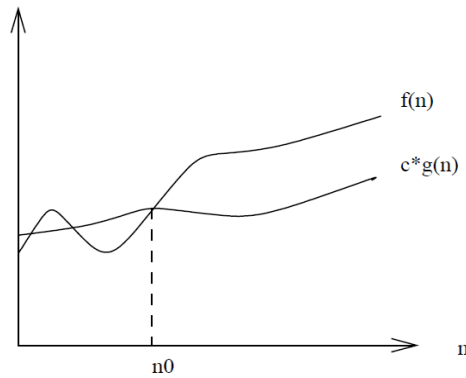
Formal Definitions

- $f(n) = O(g(n))$ if there are positive constants n_0 and c such that to the right of n_0 , the value of $f(n)$ always lies on or below $c \cdot g(n)$.
- $f(n) = \Omega(g(n))$ if there are positive constants n_0 and c such that to the right of n_0 , the value of $f(n)$ always lies on or above $c \cdot g(n)$.
- $f(n) = \Theta(g(n))$ if there exist positive constants n_0 , c_1 , and c_2 such that to the right of n_0 , the value of $f(n)$ always lies between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ inclusive.

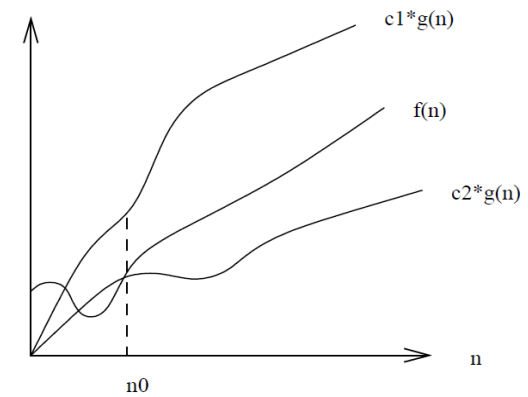
O , Ω , and Θ



(a)



(b)



(c)

The definitions imply a constant n_0 *beyond which* they are satisfied. We do not care about small values of n .

Big-oh

(19)

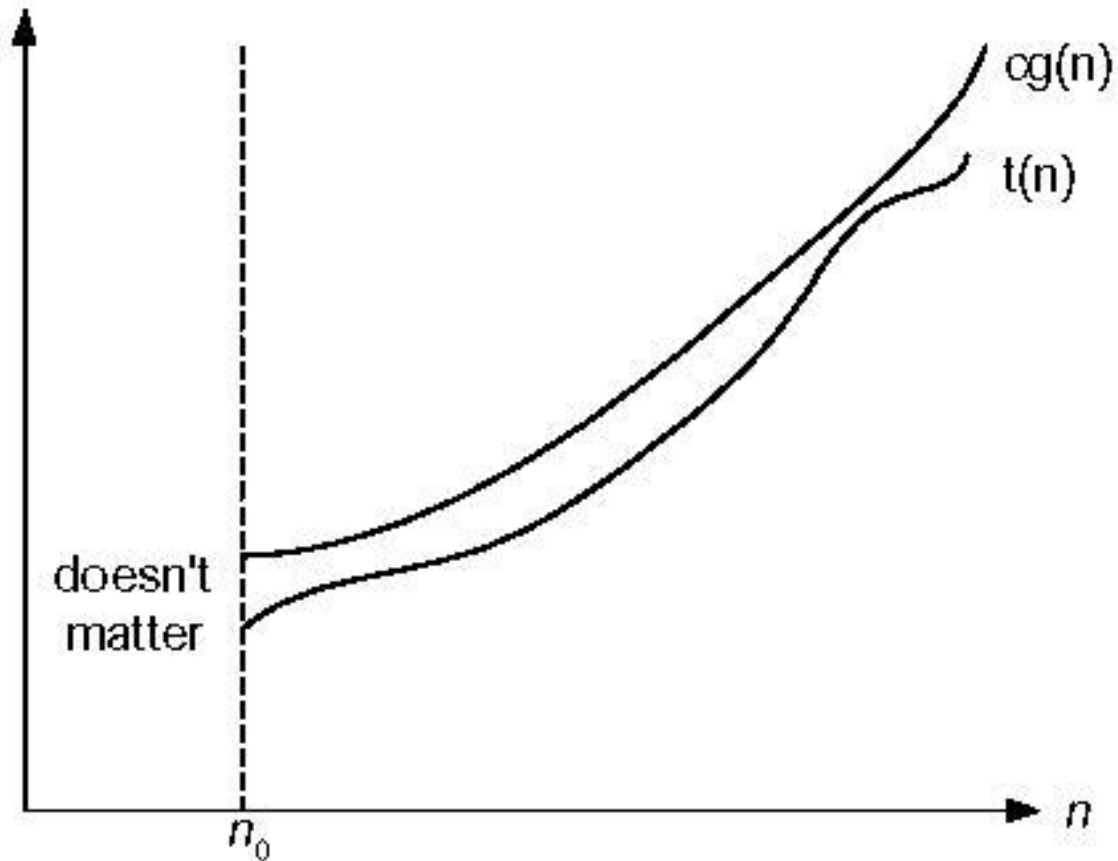


Figure 2.1 Big-oh notation: $t(n) \in O(g(n))$

Establishing order of growth using the definition

20

Definition: $f(n)$ is in $O(g(n))$ if order of growth of $f(n) \leq$ order of growth of $g(n)$ (within constant multiple), i.e., there exist positive constant c and non-negative integer n_0 such that

$$f(n) \leq c g(n) \text{ for every } n \geq n_0$$

Examples:

- $10n$ is $O(n^2)$
- $5n+20$ is $O(n)$



Big Oh Examples

$$3n^2 - 100n + 6 = O(n^2) \text{ because } 3n^2 > 3n^2 - 100n + 6$$

$$3n^2 - 100n + 6 = O(n^3) \text{ because } .01n^3 > 3n^2 - 100n + 6$$

$$3n^2 - 100n + 6 \neq O(n) \text{ because } c \cdot n < 3n^2 \text{ when } n > c$$

Think of the equality as meaning *in the set of functions*.

Some properties of **asymptotic** order of growth

22

- $f(n) \in O(f(n))$
- $f(n) \in O(g(n))$ iff $g(n) \in \Omega(f(n))$
- If $f(n) \in O(g(n))$ and $g(n) \in O(h(n))$, then $f(n) \in O(h(n))$

Note similarity with $a \leq b$

- If $f_1(n) \in O(g_1(n))$ and $f_2(n) \in O(g_2(n))$, then
$$f_1(n) + f_2(n) \in O(\max\{g_1(n), g_2(n)\})$$

Big-O Notation - Rule 1



- If an algorithm performs a certain sequence of steps $f(N)$ times for a mathematical function f , it takes $O(f(N))$ steps.

Big-O Notation - Rule 2



- If an algorithm performs an operation that takes $O(f(N))$ steps and then performs a second operation that takes $O(g(N))$ steps for functions f and g , the algorithm's total performance is $O(f(N) + g(N))$.

Big-O Notation - Rule 3



- If an algorithm takes $O(f(N) + g(N))$ and the function $f(N)$ is greater than $g(N)$ for large N , the algorithm's performance can be simplified to $O(f(N))$.

Big-O Notation - Rule 4



- If an algorithm performs an operation that takes $O(f(N))$ steps, and for every step in that operation it performs another $O(g(N))$ steps, the algorithm's total performance is $O(f(N) \times g(N))$.

Big-O Notation - Rule 5



- Ignore constant multiples. If C is a constant, $O(C \times f(N))$ is the same as $O(f(N))$, and $O(f(C \times N))$ is the same as $O(f(N))$.

Big-omega

(28)

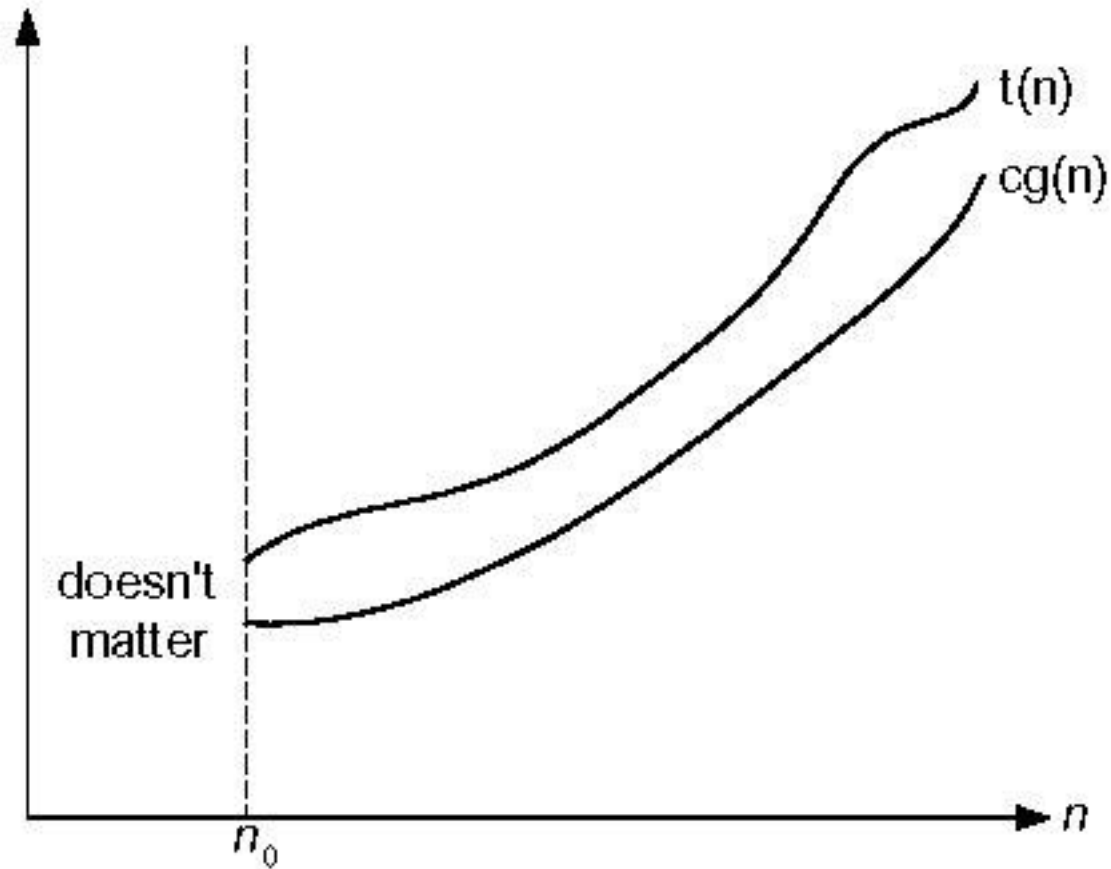


Fig. 2.2 Big-omega notation: $t(n) \in \Omega(g(n))$



Big Omega Examples

$3n^2 - 100n + 6 = \Omega(n^2)$ because $2.99n^2 < 3n^2 - 100n + 6$

$3n^2 - 100n + 6 \neq \Omega(n^3)$ because $3n^2 - 100n + 6 < n^3$

$3n^2 - 100n + 6 = \Omega(n)$ because $10^{10^{10}}n < 3n^2 - 100 + 6$

Big-theta

(30)

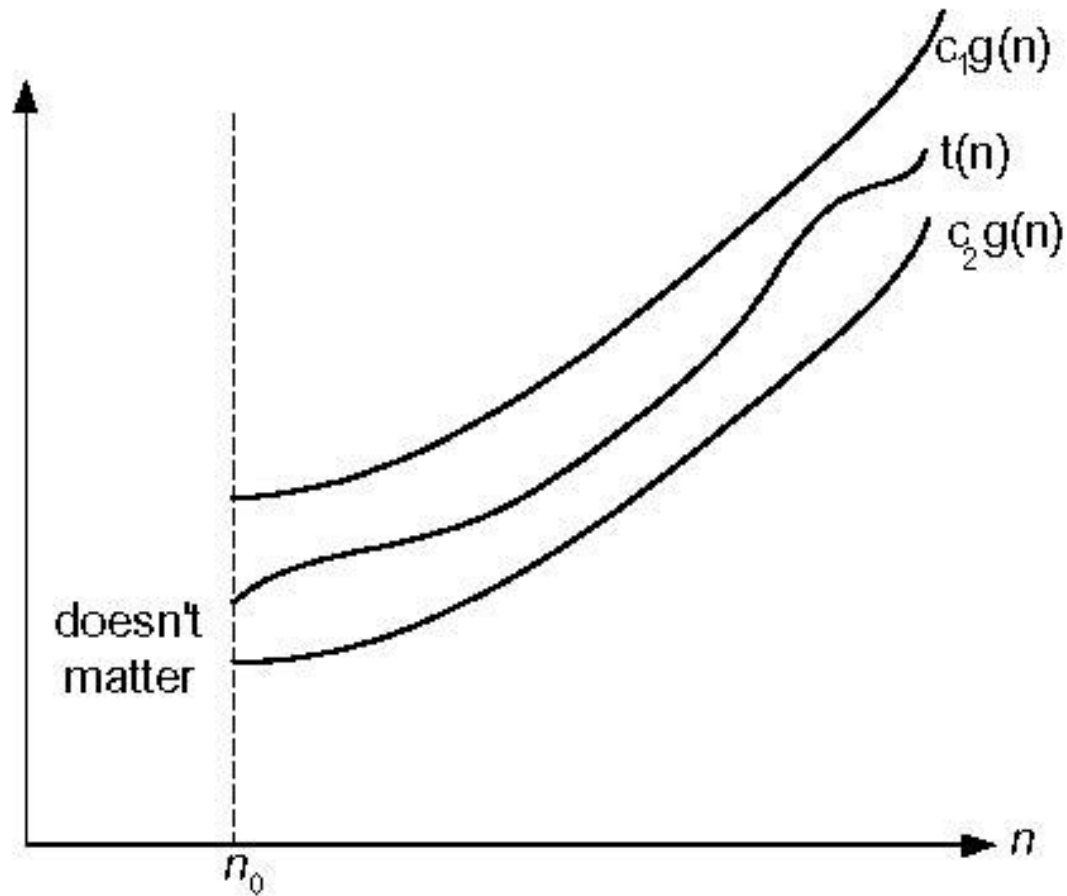


Figure 2.3 Big-theta notation: $t(n) \in \Theta(g(n))$

Big Theta Examples

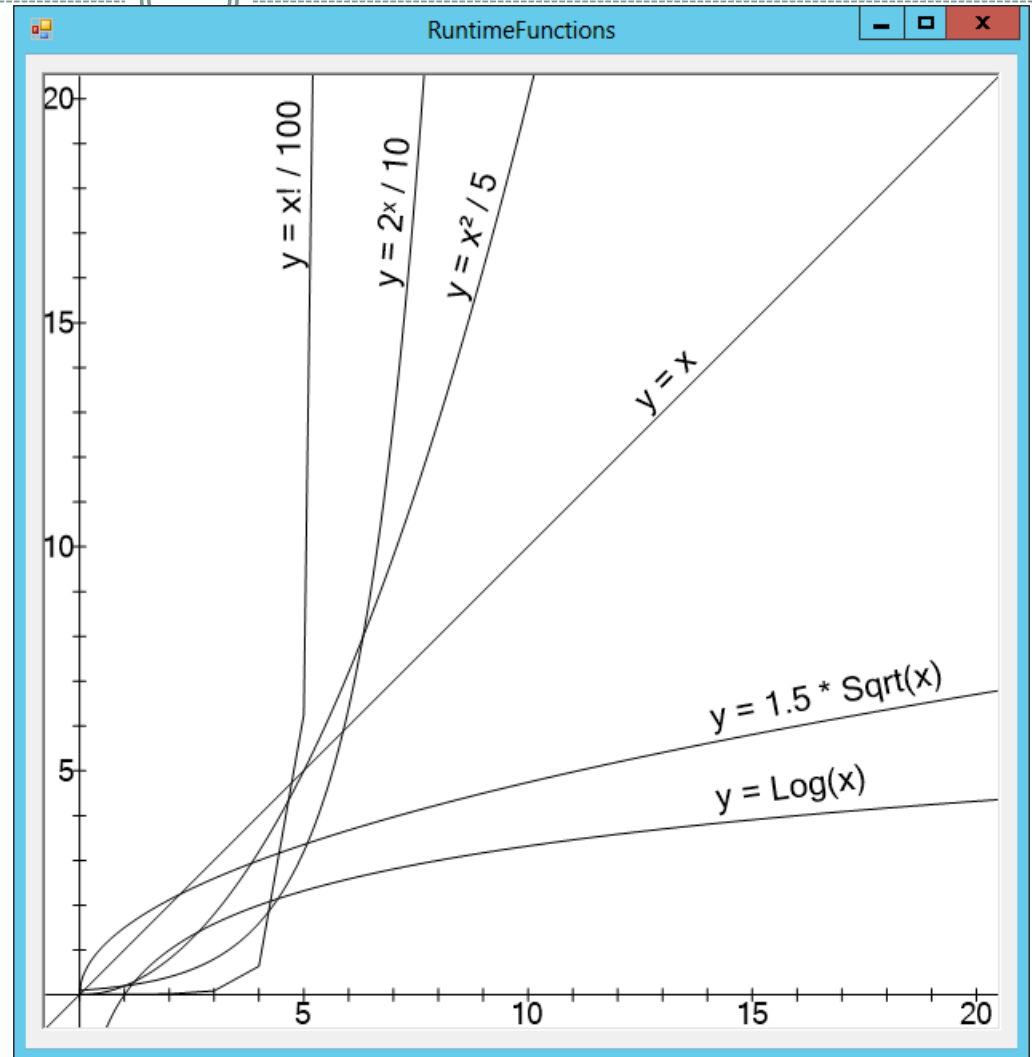
$3n^2 - 100n + 6 = \Theta(n^2)$ because O and Ω

$3n^2 - 100n + 6 \neq \Theta(n^3)$ because O only

$3n^2 - 100n + 6 \neq \Theta(n)$ because Ω only

Common Runtime Functions

- 1
- $\log N$
- $\sqrt{N}$
- N
- $N \log N$
- N^2
- 2^N
- $N!$





Asymptotic Dominance in Action

n	$f(n)$	$\lg n$	n	$n \lg n$	n^2	2^n	$n!$
10		0.003 μs	0.01 μs	0.033 μs	0.1 μs	1 μs	3.63 ms
20		0.004 μs	0.02 μs	0.086 μs	0.4 μs	1 ms	77.1 years
30		0.005 μs	0.03 μs	0.147 μs	0.9 μs	1 sec	8.4×10^{15} yrs
40		0.005 μs	0.04 μs	0.213 μs	1.6 μs	18.3 min	
50		0.006 μs	0.05 μs	0.282 μs	2.5 μs	13 days	
100		0.007 μs	0.1 μs	0.644 μs	10 μs	4×10^{13} yrs	
1,000		0.010 μs	1.00 μs	9.966 μs	1 ms		
10,000		0.013 μs	10 μs	130 μs	100 ms		
100,000		0.017 μs	0.10 ms	1.67 ms	10 sec		
1,000,000		0.020 μs	1 ms	19.93 ms	16.7 min		
10,000,000		0.023 μs	0.01 sec	0.23 sec	1.16 days		
100,000,000		0.027 μs	0.10 sec	2.66 sec	115.7 days		
1,000,000,000		0.030 μs	1 sec	29.90 sec	31.7 years		

Implications of Dominance

- Exponential algorithms get hopeless fast.
- Quadratic algorithms get hopeless at or before 1,000,000.
- $O(n \log n)$ is possible to about one billion.
- $O(\log n)$ never sweats.

Testing Dominance

$f(n)$ dominates $g(n)$ if $\lim_{n \rightarrow \infty} g(n)/f(n) = 0$, which is the same as saying $g(n) = o(f(n))$.

Note the little-oh – it means “grows strictly slower than”.



Implications of Dominance

- n^a dominates n^b if $a > b$ since

$$\lim_{n \rightarrow \infty} n^b / n^a = n^{b-a} \rightarrow 0$$

- $n^a + o(n^a)$ doesn't dominate n^a since

$$\lim_{n \rightarrow \infty} n^a / (n^a + o(n^a)) \rightarrow 1$$

Establishing order of growth using limits

37

$$\lim_{n \rightarrow \infty} T(n)/g(n) = \begin{cases} 0 & \text{order of growth of } T(n) < \text{order of growth of } g(n) \\ c > 0 & \text{order of growth of } T(n) = \text{order of growth of } g(n) \\ \infty & \text{order of growth of } T(n) > \text{order of growth of } g(n) \end{cases}$$

Examples:

• $10n$ vs. n^2

• $n(n+1)/2$ vs. n^2

• n^3 vs n^2

Orders of growth of some important functions

38

- All logarithmic functions $\log_a n$ belong to the same class $\Theta(\log n)$ no matter what the logarithm's base $a > 1$ is
- All polynomials of the same degree k belong to the same class: $a_k n^k + a_{k-1} n^{k-1} + \dots + a_0 \in \Theta(n^k)$
- Exponential functions a^n have different orders of growth for different a 's
- order $\log n < \text{order } n^\alpha \ (\alpha > 0) < \text{order } a^n < \text{order } n! < \text{order } n^n$

Basic asymptotic efficiency classes

39

1	constant
$\log n$	logarithmic
n	linear
$n \log n$	n -log- n or linearithmic
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial



Dominance Rankings

You must come to accept the dominance ranking of the basic functions:

$$n! \gg 2^n \gg n^3 \gg n^2 \gg n \log n \gg n \gg \log n \gg 1$$

Advanced Dominance Rankings

Additional functions arise in more sophisticated analysis than we will do in this course:

$$n! \gg c^n \gg n^3 \gg n^2 \gg n^{1+\epsilon} \gg n \log n \gg n \gg \sqrt{n} \gg \log^2 n \gg \log n \gg \log n / \log \log n \gg \log \log n \gg \alpha(n) \gg 1$$

Examples



True or False?

1. $2n^2 + 1 = O(n^2)$

2. $\sqrt{n} = O(\log n)$

3. $\log n = O(\sqrt{n})$

4. $n^2(1 + \sqrt{n}) = O(n^2 \log n)$

5. $3n^2 + \sqrt{n} = O(n^2)$

6. $\sqrt{n} \log n = O(n)$

7. $\log n = O(n^{-1/2})$

Take limit, if 0 or ∞ true

Else false

Analyzing Programs

43

- Program profilers are available that will indicate where the program is spending its execution time
- Can be done by hand by:
 - Incrementing a unique counter every time each subprogram is called
 - Incrementing a unique counter every time a particular code segment is reached

Analysis of algorithms

44

- **Issues:**
 - correctness
 - time efficiency
 - space efficiency
- **Approaches:**
 - theoretical analysis
 - empirical analysis

Empirical analysis of time efficiency

45

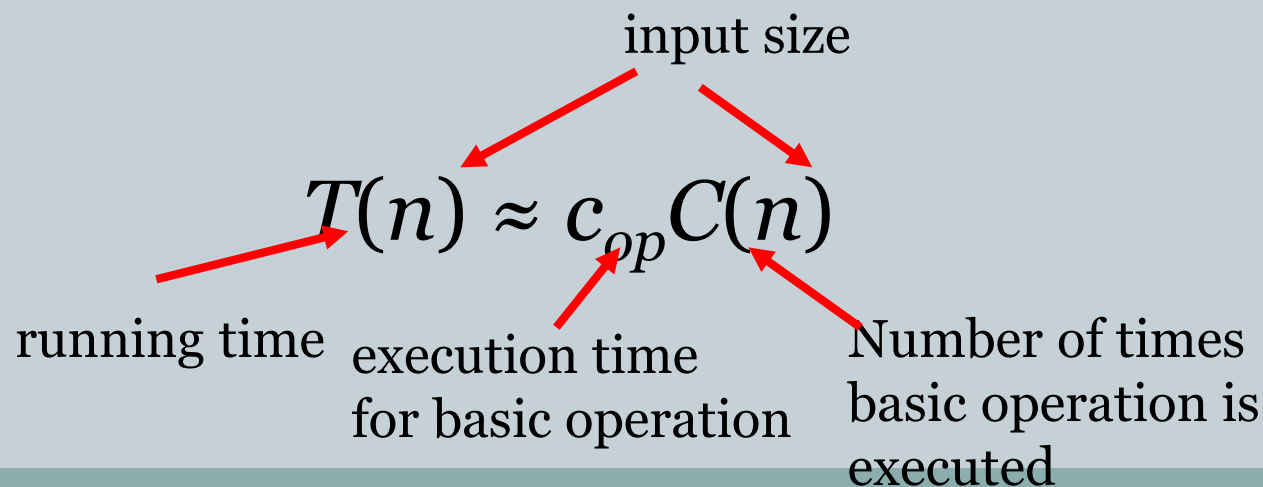
- Select a specific (typical) sample of inputs
- Use physical unit of time (e.g., milliseconds)
or
Count actual number of basic operation's executions
- Analyze the empirical data

Theoretical analysis of time efficiency

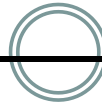
46

Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size

- Basic operation: the operation that contributes most towards the running time of the algorithm



Input size and basic operation examples



<i>Problem</i>	<i>Input size measure</i>	<i>Basic operation</i>
Searching for key in a list of n items	Number of list's items, i.e. n	Key comparison
Multiplication of two matrices	Matrix dimensions or total number of elements	Multiplication of two numbers